Aplicación del aprendizaje por refuerzo a Tetris

Edrhian Valerio Andrés

edrhian27@email.com

24 de mayo de 2025

Abstract

Entrenamiento de un agente en el videojuego *Tetris* aplicando técnicas de aprendizaje por refuerzo

1 Introducción

El aprendizaje por refuerzo es un área dentro del aprendizaje automático, que se centra en cómo un agente debe tomar acciones en un entorno dinámico con el objetivo de maximizar una señal de recompensa.

Uno de los principales avances de esta área aplicada a juegos son (Tesauro, 1995), donde se entrenó un agente en un entorno de backgammon. También se ha utilizado para entornos de Atari con inputs visuales, y otros como ajedrez, Go, incluso videojuegos complejos como (OpenAI, 2018).

En cuanto a *Tetris*, ha habido enfoques diferentes donde varía el entorno, en enfoques como (Cox, 2019), la acción del agente es un cambio completo de la posición de la pieza, es decir, no utiliza los controles humanos, como desplazar o rotar la pieza, sino que se calcula cada posición final posible de la pieza dentro del tablero, sin tener en cuenta los movimientos requeridos para esa posición.

El principal enfoque de este trabajo es entrenar a un agente en un entorno de *Tetris* moderno, similar a (Eddie & contributors, s.f.), y conseguir un rendimiento superior a sus enfoques previos, en términos de puntuación y velocidad.

2 Desarrollo

2.1 Entorno

Se ha desarrollado un entorno parecido al *Tetris* moderno, algunas características han sido suprimidas debido a la lógica de programación del entorno, como los movimientos avanzados de piezas como ("List of twists", s.f.). El agente puede hacer las mismas acciones que el juego moderno (Eddie & contributors, s.f.).

Se ha experimentado con dos tipos de observación, el primero es una observación visual con el tablero, añadiendo un pre-proceso explicado en el punto 2.2. El segundo tipo de observación son datos de un diccionario, donde se devuelve información como el tablero sin la pieza activa, la siguiente pieza, rotación, etc.

Las recompensas se calculan cada vez que la pieza activa se bloquea dentro del tablero.

$$\text{reward} = \mathit{cleared_lines} + \alpha \cdot \mathit{stack_height} - \mathit{holes} - \mathit{cheese_layers} - \mathit{bumpiness} + \beta \cdot \mathit{finesse}$$

$$\text{donde } \alpha, \beta \in \{-1, +1\} \text{ según el contexto.}$$

cleared_lines aumenta la recompensa de forma exponencial por el número de líneas completadas, *stack_height* determina que tan alto está colocando las piezas, *holes* es el número de agujeros tapados por una pieza, *cheese layers* es la suma de la cantidad de bloques encima de cada agujero, *bumpiness* es la suma de la diferencia de la altura entre columnas y *finesse* es la cantidad de movimientos por pieza.

2.2 Modelos

Se han utilizado dos modelos diferentes para el entrenamiento del agente.

El primero es una red neuronal completamente conectada (*fully connected neural network*), como se muestra en la Figura 1, con capas de normalización para mejorar la estabilidad.

El segundo modelo es una red neuronal convolucional como la Figura 2, con la misma arquitectura propuesta por (Mnih et al., 2013). Cada fotograma del juego tiene una resolución de 660 x 500 y 3

canales RGB. Para reducir el coste computacional, se aplicó un preprocesamiento a cada fotograma: se recortó para ajustar solo el área del tablero dejando una dimensión de 660 x 300, se convirtió a escala de grises, se redimensionó el tamaño a 84 x 84 y se normalizaron los valores. Este proceso se repite en cada *timestep* y se apilan los últimos cuatro fotogramas del juego.

Figure 1: Arquitectura de la DQN con estados tabulares

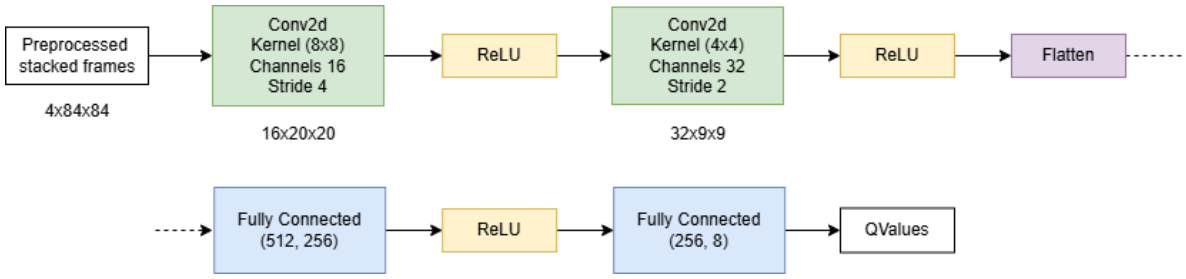

Figure 2: Arquitectura de la DQN con estados visuales

2.3 Algoritmo

De forma similar a (Mnih et al., 2013), se utilizó el algoritmo de *Deep Q Learning* con *Replay Memory*, añadiendo un soft update para mejorar la estabilidad, el uso de la función de pérdida de *smoothL1Loss* y se aplicó *gradient clipping* para evitar gradientes demasiado altos.

3 Resultados

Cada modelo se entrenó con un valor inicial de $\epsilon = 1$ que decaía hasta $\epsilon = 0.1$ en un rango de uno a dos millones de *timesteps*, con un factor de descuento de $\gamma = 0.99$, un factor de actualización de $\tau = 0.001$ y una tasa de aprendizaje de $\alpha = 1e^{-4}$.

En cuanto a los modelos de observación tabular, se entrenaron dos modelos distintos, uno con la visión limitada a las seis capas más altas del juego que contenían alguna pieza y otro con observación de la altura de cada columna del tablero, ambos un *Replay Memory* con capacidad de 250 mil transiciones. Luego, se entrenó un solo modelo convolucional con un *Replay Memory* de 25 mil transiciones.

En cada modelo analizamos la supervivencia y la eficiencia del agente a través de la media de piezas colocadas y líneas completadas en una partida, cada modelo se entrenó por 16 mil episodios.

Modelo	Piezas Colocadas	Líneas Completadas
6 Layer View	27.19	0.16
Column Height Index	28.98	0.19
CNN	16.80	0.16

Table 1: Promedio de piezas colocadas y líneas completadas de cada modelo después de mil episodios

4 Conclusiones

Los modelos entrenados no han logrado un rendimiento superior a sus enfoques previos.

La principal dificultad en conseguir la convergencia se encuentra en el tiempo entre la recompensa y las acciones, el agente debe aprender cuál es la cadena de acciones necesarias para que una pieza acabe en un estado beneficioso.

Para mejorar los resultados se debería cambiar la información que se provee al agente sobre el estado y la cantidad de episodios de exploración. Además, se debería reconsiderar la arquitectura del modelo convolucional, ya que podría no estar detectando bien las características relevantes del tablero. Un factor clave de la optimización está en ajustar las recompensas o penalizaciones para evitar valores desproporcionadamente altos o bajos que dificulten la optimización. Finalmente, se debería probar otros algoritmos como *Advantage Actor Critic* o *Proximal Policy Optimization*.

Bibliografia

Berner, C., Brockman, G., Chan, B., Cheung, V., Dębiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., de Oliveira Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., ... Zhang, S. (2021). *Dota 2 with large scale deep reinforcement learning*. https://arxiv.org/pdf/1912.06680

Bullet, C. (2020). *Tetris-ai-javascript*. https://github.com/Code-Bullet/Tetris-AI-Javascript

Carr, D. (2005). *Adapting reinforcement learning to tetris*. https://www.cs.ru.ac.za/research/g02c0108/files/finalwriteuphandin.pdf

Code, J. (2025). *Gym_solutions*. https://github.com/johnnycode8/gym_solutions

Cox, M. (2019). *Tetris-dqn*. https://github.com/michiel-cox/Tetris-DQN

Eddie & contributors. (s.f.). *Jstris guide*. https://jstris.jezevec10.com/guide#controls

Farama Foundation. (2025). *Gymnasium: An api standard for reinforcement learning with a diverse collection of reference environments*. https://gymnasium.farama.org/

Lee, Y. (2013). *Tetris ai - the (near) perfect bot*. https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/

List of twists. (s.f.). https://harddrop.com/wiki/List_of_twists

Liu, H., & Liu, L. (2020). *Learn to play tetris with deep reinforcement learning*. https://openreview.net/pdf?id=8TLyqLGQ7Tg

Lundgaard, N., & Mckee, B. (2006). *Reinforcement learning and neural networks for tetris*. https://www.idi.ntnu.no/emner/it3105/materials/neural/lundgaard-tetris-ann.pdf

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). *Playing atari with deep reinforcement learning*. https://arxiv.org/pdf/1312.5602

OpenAI. (2018). *Spinning up in deep reinforcement learning*. https://spinningup.openai.com/en/latest/index.html

OpenAI. (2025). *Chatgpt (versión gpt-4.1 mini) [large language model]*. https://chat.openai.com/chat

Paszke, A. (2024). *Reinforcement learning dqn tutorial*. https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html

Simonini, T., Sanseveiro, O., & Paul, S. (2025). *Deep reinforcement learning course*. https://huggingface.co/learn/deep-rl-course

Stevens, M., & Pradhan, S. (2016). *Playing tetris with deep reinforcement learning*. https://cs231n.stanford.edu/reports/2016/pdfs/121_Report.pdf

Sutton, R., & Barto, A. (2020). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/RLbook2020.pdf

Tesauro, G. (1995). Temporal difference learning and td-gammon. *Communications of the ACM*. https://dl.acm.org/doi/pdf/10.1145/203330.203343

Thiery, C., & Scherrer, B. (2009). Building controllers for tetris. *International Computer Games Association Journal*, *32*(1), 3–11. https://inria.hal.science/inria-00418954/document

zeroize318. (2021). *Tetris_ai*. https://github.com/zeroize318/tetris_ai